ОТЧЁТ

по лабораторной работе №9

«Сетевое взаимодействие в Python: TCP/UDP и прикладной протокол»

Вариант 20. Интернет-провайдер, сущность Tariff

1. Цель работы

Научиться строить клиент-серверное приложение на сокетах: определить свой прикладной протокол, обслужить нескольких клиентов по TCP, обработать ошибки и тайм-аут и показать справочный запрос по UDP.

2. Постановка индивидуального задания

Вариант 20. Основная сущность — тариф, специализированная команда — поиск тарифов по скорости. Клиент умеет получить список, один объект, создать, изменить и удалить тариф, а также выполнить поиск.

Сообщения — одна строка JSON. Сервер пишет журнал, для каждого клиента запускает поток и ограничивает ожидание тайм-аутом. UDP-поиск используется как справочная операция, для которой потеря датаграммы допустима.

3. Краткие теоретические сведения

Сокет — конечная точка обмена по сети. TCP устанавливает соединение и гарантирует порядок байтов, но recv может вернуть только часть сообщения, поэтому граница сообщения задаётся символом перевода строки. UDP отправляет отдельные датаграммы и доставку не обещает.

Прикладной протокол поверх транспорта договаривается о командах и ответах. В работе команда содержит поле cmd, успешный ответ — ok и data, ошибка — ok=false и текст. Своё исключение ProtocolError отделяет порчу формата от ошибок предметной области.

4. Описание программы

Сервер слушает 127.0.0.1:19020 для TCP и 19021 для UDP. При старте в памяти уже есть три тарифа. Команды list, get, create, update, delete, search_speed и ping обрабатывает один и тот же метод, поэтому TCP и UDP не расходятся в правилах. Неизвестный идентификатор возвращает ошибку, а не обрывает сервер.

Клиент показывает меню и печатает JSON-ответ. Поиск по скорости — специализированная команда варианта. Журнал сервера пишет подключение, команду и отключение. Тесты поднимают сервер на отдельных портах и проверяют поиск, отсутствующий тариф и одновременную работу двух клиентов.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — client.py

```
"""
TCP-клиент тарифов. Вариант 20: специализированная команда — поиск по скорости.
"""


import json
import socket
import sys

from protocol import ProtocolError, decode, encode

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")

HOST = "127.0.0.1"
TCP_PORT = 19020
UDP_PORT = 19021



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



class TariffClient:
    def __init__(self, host=HOST, port=TCP_PORT, timeout=3):
        self.host = host
        self.port = port
        self.timeout = timeout
        self.sock = None

    def connect(self):
        self.sock = socket.create_connection((self.host, self.port), timeout=self.timeout)
        self.sock.settimeout(self.timeout)

    def request(self, payload: dict) -> dict:
        if self.sock is None:
            raise ProtocolError("нет соединения")
        self.sock.sendall(encode(payload))
        buffer = ""
        while "\n" not in buffer:
            try:
                chunk = self.sock.recv(4096)
            except socket.timeout as exc:
                raise TimeoutError("сервер не ответил за отведённое время") from exc
            if not chunk:
```

```
                raise ConnectionError("сервер закрыл соединение")
            buffer += chunk.decode("utf-8")
        line = buffer.split("\n", 1)[0]
        return decode(line)

    def close(self):
        if self.sock is not None:
            self.sock.close()
            self.sock = None


def udp_search(min_speed: int) -> dict:
    payload = encode({"cmd": "search_speed", "min_speed": min_speed})
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.settimeout(2)
        sock.sendto(payload.strip(), (HOST, UDP_PORT))
        data, _ = sock.recvfrom(8192)
        return json.loads(data.decode("utf-8"))


def show(response):
    print(json.dumps(response, ensure_ascii=False, indent=2))


def print_menu():
    print()
    print("===== Клиент тарифов =====")
    print("1. Получить список")
    print("2. Получить объект")
    print("3. Создать")
    print("4. Изменить")
    print("5. Удалить")
    print("6. Поиск тарифов по скорости")
    print("7. Ping")
    print("8. UDP-поиск по скорости")
    print("0. Выход")
    print()


def main():
    client = TariffClient()
    try:
        client.connect()
    except OSError as exc:
        print(f"Не удалось подключиться к {HOST}:{TCP_PORT}: {exc}")
        print("Сначала запустите server.py")
        return
    print(f"Подключено к {HOST}:{TCP_PORT}")

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                show(client.request({"cmd": "list"}))
            elif choice == "2":
                item_id = int(read_line("ID: "))
                show(client.request({"cmd": "get", "id": item_id}))
            elif choice == "3":
                name = read_line("Название: ")
                speed = int(read_line("Скорость: "))
                price = float(read_line("Цена: ").replace(",", "."))
                region = read_line("Регион: ")
                show(client.request({
                    "cmd": "create", "name": name, "speed": speed,
                    "price": price, "region": region,
                }))
            elif choice == "4":
                item_id = int(read_line("ID: "))
                price = float(read_line("Новая цена: ").replace(",", "."))
                show(client.request({"cmd": "update", "id": item_id, "price": price}))
            elif choice == "5":
                item_id = int(read_line("ID: "))
                show(client.request({"cmd": "delete", "id": item_id}))
```

```
            elif choice == "6":
                speed = int(read_line("Минимальная скорость: "))
                show(client.request({"cmd": "search_speed", "min_speed": speed}))
            elif choice == "7":
                show(client.request({"cmd": "ping"}))
            elif choice == "8":
                speed = int(read_line("Минимальная скорость: "))
                print("UDP не гарантирует доставку. Для справочного поиска это допустимо:")
                print("повторный запрос дешевле, чем держать соединение учёта тарифов.")
                show(udp_search(speed))
            elif choice == "0":
                print("Работа клиента завершена.")
                break
            else:
                print("Ошибка: выберите пункт от 0 до 8.")
        except (ValueError, ProtocolError, TimeoutError, ConnectionError, OSError) as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            break
    client.close()


if __name__ == "__main__":
    main()
```

Листинг — models.py

```
"""Тариф интернет-провайдера."""

from dataclasses import dataclass, asdict


@dataclass
class Tariff:
    id: int
    name: str
    speed: int
    price: float
    region: str

    def to_dict(self):
        return asdict(self)

    @classmethod
    def from_dict(cls, data):
        return cls(
            id=int(data["id"]),
            name=str(data["name"]),
            speed=int(data["speed"]),
            price=float(data["price"]),
            region=str(data["region"]),
        )
```

Листинг — protocol.py

```
"""Прикладной протокол: одно JSON-сообщение на строку."""

import json


class ProtocolError(Exception):
    """Собственное исключение протокола."""


def encode(payload: dict) -> bytes:
    text = json.dumps(payload, ensure_ascii=False)
    return (text + "\n").encode("utf-8")


def decode(raw: str) -> dict:
    try:
        data = json.loads(raw)
    except json.JSONDecodeError as exc:
```

```
        raise ProtocolError("ожидался JSON") from exc
    if not isinstance(data, dict):
        raise ProtocolError("ожидался JSON-объект")
    if "cmd" not in data and "ok" not in data:
        raise ProtocolError("в сообщении нет поля cmd")
    return data


def ok(data):
    return {"ok": True, "data": data}


def fail(message):
    return {"ok": False, "error": message}
```

Листинг — repository.py

```
"""Хранилище тарифов в памяти."""

from models import Tariff


class TariffRepository:
    def __init__(self):
        self._items: dict[int, Tariff] = {}
        self._next = 1

    def seed(self):
        self.add("Домашний 100", 100, 500, "Москва")
        self.add("Домашний 300", 300, 800, "Москва")
        self.add("Корпоративный 500", 500, 2500, "Казань")

    def add(self, name, speed, price, region) -> Tariff:
        item = Tariff(self._next, name, speed, price, region)
        self._items[item.id] = item
        self._next += 1
        return item

    def all(self) -> list[Tariff]:
        return list(self._items.values())

    def get(self, item_id: int) -> Tariff | None:
        return self._items.get(item_id)

    def update(self, item_id: int, **fields) -> Tariff | None:
        item = self._items.get(item_id)
        if item is None:
            return None
        for key, value in fields.items():
            if value is not None and hasattr(item, key):
                setattr(item, key, value)
        return item

    def delete(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None

    def by_speed(self, min_speed: int) -> list[Tariff]:
        return [item for item in self._items.values() if item.speed >= min_speed]
```

Листинг — server.py

```
"""
TCP-сервер тарифов. Несколько клиентов обслуживаются в отдельных потоках.
UDP-порт отвечает на поиск по скорости: для справочного запроса потеря пакета допустима.
"""

import logging
import os
import socket
import sys
import threading
```

```
from protocol import ProtocolError, decode, encode, fail, ok
from repository import TariffRepository

BASE = os.path.dirname(os.path.abspath(__file__))
LOG_PATH = os.path.join(BASE, "server.log")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    handlers=[
        logging.FileHandler(LOG_PATH, encoding="utf-8"),
        logging.StreamHandler(sys.stdout),
    ],
)
log = logging.getLogger("tariff-server")



class TariffServer:
    def __init__(self, host="127.0.0.1", tcp_port=19020, udp_port=19021):
        self.host = host
        self.tcp_port = tcp_port
        self.udp_port = udp_port
        self.repository = TariffRepository()
        self.repository.seed()
        self._stop = threading.Event()

    def handle_command(self, message: dict) -> dict:
        cmd = message.get("cmd")
        try:
            if cmd == "ping":
                return ok("pong")
            if cmd == "list":
                return ok([item.to_dict() for item in self.repository.all()])
            if cmd == "get":
                item = self.repository.get(int(message["id"]))
                if item is None:
                    return fail("тариф не найден")
                return ok(item.to_dict())
            if cmd == "create":
                item = self.repository.add(
                    message["name"],
                    int(message["speed"]),
                    float(message["price"]),
                    message.get("region", "Москва"),
                )
                return ok(item.to_dict())
            if cmd == "update":
                fields = {}
                for key in ("name", "speed", "price", "region"):
                    if key in message:
                        fields[key] = message[key]
                if "speed" in fields:
                    fields["speed"] = int(fields["speed"])
                if "price" in fields:
                    fields["price"] = float(fields["price"])
                item = self.repository.update(int(message["id"]), **fields)
                if item is None:
                    return fail("тариф не найден")
                return ok(item.to_dict())
            if cmd == "delete":
                if not self.repository.delete(int(message["id"])):
                    return fail("тариф не найден")
                return ok("удалён")
            if cmd == "search_speed":
                found = self.repository.by_speed(int(message["min_speed"]))
                return ok([item.to_dict() for item in found])
            return fail("неизвестная команда")
        except (KeyError, TypeError, ValueError) as exc:
            return fail(f"некорректные поля: {exc}")

    def serve_client(self, conn: socket.socket, address):
        log.info("Client connected: %s:%s", address[0], address[1])
        conn.settimeout(30)
        buffer = ""
        try:
```

```
            while not self._stop.is_set():
                try:
                    chunk = conn.recv(4096)
                except socket.timeout:
                    log.info("Timeout: %s:%s", address[0], address[1])
                    conn.sendall(encode(fail("тайм-аут ожидания команды")))
                    break
                if not chunk:
                    break
                buffer += chunk.decode("utf-8")
                while "\n" in buffer:
                    line, buffer = buffer.split("\n", 1)
                    if not line.strip():
                        continue
                    log.info("Command: %s", line.strip()[:120])
                    try:
                        message = decode(line)
                        response = self.handle_command(message)
                    except ProtocolError as exc:
                        response = fail(str(exc))
                    conn.sendall(encode(response))
        except OSError as exc:
            log.info("Client error %s: %s", address, exc)
        finally:
            conn.close()
            log.info("Client disconnected: %s:%s", address[0], address[1])

    def tcp_loop(self):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
            server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            server.bind((self.host, self.tcp_port))
            server.listen()
            server.settimeout(0.5)
            log.info("TCP %s:%s", self.host, self.tcp_port)
            while not self._stop.is_set():
                try:
                    conn, address = server.accept()
                except socket.timeout:
                    continue
                thread = threading.Thread(target=self.serve_client, args=(conn, address), daemon=True)
                thread.start()

    def udp_loop(self):
        """Справочный UDP-запрос. Потеря датаграммы не ломает учёт тарифов."""
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
            sock.bind((self.host, self.udp_port))
            sock.settimeout(0.5)
            log.info("UDP %s:%s", self.host, self.udp_port)
            while not self._stop.is_set():
                try:
                    data, address = sock.recvfrom(4096)
                except socket.timeout:
                    continue
                message = {}
                try:
                    message = decode(data.decode("utf-8"))
                    response = self.handle_command(message)
                except ProtocolError as exc:
                    response = fail(str(exc))
                log.info("UDP %s from %s", message.get("cmd", "?"), address)
                sock.sendto(encode(response).strip(), address)

    def start(self):
        threading.Thread(target=self.tcp_loop, daemon=True).start()
        threading.Thread(target=self.udp_loop, daemon=True).start()

    def stop(self):
        self._stop.set()


def main():
    server = TariffServer()
    log.info("Сервер тарифов запущен. Остановка: Ctrl+C")
    server.start()
    try:
```

```
        threading.Event().wait()
    except KeyboardInterrupt:
        server.stop()
        log.info("Сервер остановлен")


if __name__ == "__main__":
    main()
```

Листинг — tests\test_protocol.py

```
import os
import socket
import sys
import threading
import time

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from protocol import decode, encode
from server import TariffServer



def test_roundtrip():
    server = TariffServer(tcp_port=19120, udp_port=19121)
    server.start()
    time.sleep(0.3)
    try:
        with socket.create_connection(("127.0.0.1", 19120), timeout=2) as sock:
            sock.sendall(encode({"cmd": "search_speed", "min_speed": 300}))
            raw = sock.recv(4096).decode("utf-8")
        response = decode(raw.strip())
        assert response["ok"] is True
        speeds = [item["speed"] for item in response["data"]]
        assert speeds == [300, 500]
    finally:
        server.stop()



def test_missing_tariff():
    server = TariffServer(tcp_port=19122, udp_port=19123)
    server.start()
    time.sleep(0.3)
    try:
        with socket.create_connection(("127.0.0.1", 19122), timeout=2) as sock:
            sock.sendall(encode({"cmd": "get", "id": 99}))
            raw = sock.recv(4096).decode("utf-8")
        response = decode(raw.strip())
        assert response["ok"] is False
    finally:
        server.stop()



def test_two_clients():
    server = TariffServer(tcp_port=19124, udp_port=19125)
    server.start()
    time.sleep(0.3)
    results = []

    def worker():
        with socket.create_connection(("127.0.0.1", 19124), timeout=2) as sock:
            sock.sendall(encode({"cmd": "ping"}))
            results.append(sock.recv(1024).decode("utf-8"))

    threads = [threading.Thread(target=worker) for _ in range(2)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    server.stop()
    assert len(results) == 2
    assert all("pong" in item for item in results)
```

6. Примеры работы программы

Снимки показывают список тарифов, поиск по скорости, создание записи, UDP-запрос и автоматические тесты протокола.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python9> python client.py

Подключено к 127.0.0.1:19020

===== Клиент тарифов =====
1. Получить список
2. Получить объект
3. Создать
4. Изменить
5. Удалить
6. Поиск тарифов по скорости
7. Ping
8. UDP-поиск по скорости
0. Выход

Выберите действие: 1
{
  "ok": true,
  "data": [
    {
      "id": 1,
      "name": "Домашний 100",
      "speed": 100,
      "price": 500,
      "region": "Москва"
    },
    {
      "id": 2,
      "name": "Домашний 300",
      "speed": 300,
      "price": 800,
      "region": "Москва"
    },
    {
      "id": 3,
      "name": "Корпоративный 500",
      "speed": 500,
      "price": 2500,
      "region": "Казань"
    },
    {
      "id": 4,
      "name": "Ночной 100",
      "speed": 100,
      "price": 300.0,
      "region": "Тула"
    }
  ]
}
```

Рисунок 1 – TCP-клиент: список тарифов

```
C:\\Users\\Shmurdyakich\\Desktop\\pythonALL\\python9> python -m pytest -q

...                                                                      [100%]
3 passed in 0.94s
```

Рисунок 2 – Тесты протокола, отсутствующего тарифа и двух клиентов

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python9> python client.py

Подключено к 127.0.0.1:19020

===== Клиент тарифов =====
1. Получить список
2. Получить объект
3. Создать
4. Изменить
5. Удалить
6. Поиск тарифов по скорости
7. Ping
8. UDP-поиск по скорости
0. Выход

Выберите действие: 6
Минимальная скорость: 300
{
  "ok": true,
  "data": [
    {
      "id": 2,
      "name": "Домашний 300",
      "speed": 300,
      "price": 800,
      "region": "Москва"
    },
    {
      "id": 3,
      "name": "Корпоративный 500",
      "speed": 500,
      "price": 2500,
      "region": "Казань"
    }
  ]
}

===== Клиент тарифов =====
1. Получить список
2. Получить объект
3. Создать
4. Изменить
5. Удалить
6. Поиск тарифов по скорости
7. Ping
8. UDP-поиск по скорости
0. Выход

Выберите действие: 3
Название: Ночной 100
```

Рисунок 3 – Поиск по скорости, создание тарифа и UDP-запрос

7. Ответы на контрольные вопросы

1. Чем TCP отличается от UDP?

TCP — поток с соединением и гарантией доставки. UDP — отдельные датаграммы без такой гарантии.

2. Почему одного recv недостаточно для одного сообщения?

TCP отдаёт байты, а не готовые сообщения. Часть JSON может прийти следующим вызовом, поэтому ответ собирается до символа новой строки.

3. Зачем тайм-аут?

Без него клиент или сервер может ждать вечно, если вторая сторона молчит. Тайм-аут ограничивает ожидание и освобождает соединение.

4. Почему поиск по UDP допустим?

Это справочный запрос. Если датаграмма потеряется, клиент может повторить её. Учёт тарифов при этом остаётся на TCP.

5. Как обслуживаются несколько клиентов?

После accept каждый клиент обрабатывается в своём потоке, а репозиторий общий. Тест одновременно получает pong от двух соединений.

6. Что пишется в журнал?

Время, подключение клиента, текст команды и отключение. Журнал ведёт стандартный модуль logging.

8. Вывод

В лабораторной работе №9 тарифы варианта 20 доступны по собственному JSON-протоколу. TCP-сервер в потоках выполняет команды списка, чтения, создания, изменения, удаления и поиска по скорости. UDP оставлен для справочного поиска, ошибки формата и отсутствующий тариф возвращаются ответом, а не аварийным завершением.